

ByteBuf之二

ByteBuf的创建

北京理工大学计算机学院
金旭亮



概述

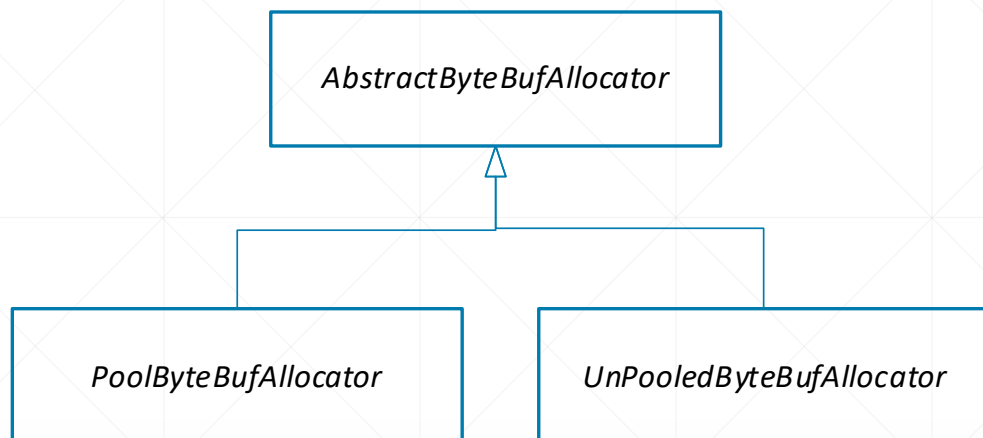
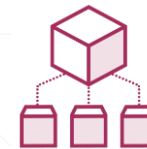


前一讲介绍了ByteBuf的基础知识，这些只是为使用ByteBuf扫清障碍，我们的真正目的，是学会使用ByteBuf。



使用ByteBuf的第一步，是创建ByteBuf实例，依据ByteBuf的类型，选择不同的字节缓冲区分配器创建。

字节缓冲区分配器



Netty默认分配器

作为数据的容器，ByteBuf需要占用一定的内存空间，为此，Netty提供了相应的数据缓冲区分配器，负责分配内存。

有两种类型的数据缓冲区分配器，一种是“池化（pool）”的，另一种是“非池化（unpooled）”的。

想一想JDK中“线程池”的概念，不难理解“池化（pool）”的作用，就是在池中保存对象，通过重用，减少对象创建与销毁所带来的开销。



在Android应用中，默认采用UnPooledByteBufAllocator，其它类型的应用中，默认使用PoolByteAllocator。

分配器示例

CreateByteBuf.java示例，展示了Netty中可用的几个分配器的使用方法。在实际开发中，通常很少直接使用字节缓冲区分配器，而是使用Netty提供的一个名为Unpooled的工具类来创建ByteBuf实例。

```
//看看默认的分配器是哪个?  
//PooledByteBufAllocator(directByDefault: true)  
System.out.println(ByteBufAllocator.DEFAULT);  
  
ByteBuf buffer = null;  
  
//方法1: 通过默认分配器分配  
//初始容量为9、最大容量为100的缓冲区  
buffer = ByteBufAllocator.DEFAULT.buffer(9, 100);  
//PooledUnsafeDirectByteBuf(ridx: 0, widx: 0, cap: 9/100)  
System.out.println(buffer);  
  
//方法2: 通过默认分配器分配  
//初始容量为256、最大容量为Integer.MAX_VALUE的缓冲区  
buffer = ByteBufAllocator.DEFAULT.buffer();  
//PooledUnsafeDirectByteBuf(ridx: 0, widx: 0, cap: 256)  
System.out.println(buffer);
```

```
//方法3: 非池化分配器，分配Java的堆 (Heap) 结构内存缓冲区  
buffer = UnpooledByteBufAllocator.DEFAULT.heapBuffer();  
//UnpooledByteBufAllocator$InstrumentedUnpooledUnsafeHeapByteBuf(ridx: 0, widx: 0, cap: 256)  
System.out.println(buffer);  
  
//方法4: 池化分配器，分配由操作系统管理的直接内存缓冲区  
buffer = PooledByteBufAllocator.DEFAULT.directBuffer();  
//PooledUnsafeDirectByteBuf(ridx: 0, widx: 0, cap: 256)  
System.out.println(buffer);
```

使用直接内存与堆内内存的ByteBuffer

类型	说明	优点	不足
Heap ByteBuffer (堆缓冲区)	内部数据为一个Java数组，存储在JVM的堆空间中，可以通过hasArray方法来判断是不是堆缓冲区	未使用池化的情况下，能快速的分配和释放	写入底层传输通道之间，需要将数据复制到直接缓冲区
Direct ByteBuffer (直接缓冲区)	内部数据存储在操作系统物理内存中	写入传输通道快速	需要进行系统调用，创建和销毁代价高昂，应该尽量将其池化。Java中读取数据时，需要将其复制到堆上。

- Heap ByteBuffer通过调用分配器的buffer()方法来创建；Direct ByteBuffer通过调用分配器的directBuffer()方法来创建。
- Heap ByteBuffer缓冲区可以直接通过array()方法读取内部数组；Direct ByteBuffer缓冲区不能读取内部数组。
- 可以调用hasArray()方法来判断是否为Heap ByteBuffer类型的缓冲区；如果hasArray()返回值为true，则表示是堆缓冲，否则为直接内存缓冲区。
- 从Direct ByteBuffer读取缓冲数据进行Java程序处理时，相对比较麻烦，需要通过getBytes、readBytes等方法先将数据复制到Java的堆内存，然后进行其他的计算。

介绍Unpooled工具类



这个类在实际开发中用得很多，主要提供以下三个功能：

- ▶ 创建各种类型的字节缓冲区实例
- ▶ 创建包装的字节缓冲区实例
- ▶ 复制现有的字节数组、字节缓冲区和字符串



Unpooled，顾名思义，这个类所创建的所有ByteBuf，都是“非池化”的，多数情况下可以自动释放内存，但在某些场景中，可能需要单独写代码手动释放内存。

使用Unpooled创建“非池化的”ByteBuf



Unpooled类干的活，就是不做任何缓存，每次调用此类所提供的方法，得到的都是一个“全新创建出来的”ByteBuf对象，几个常用的静态方法如下所示：

```
import static io.netty.buffer.Unpooled.*;

ByteBuf heapBuffer      = buffer(128);
ByteBuf directBuffer    = directBuffer(256);
ByteBuf wrappedBuffer   = wrappedBuffer(new byte[128], new byte[256]);
ByteBuf copiedBuffer    = copiedBuffer(ByteBuffer.allocate(128));
```



特别注意一下，上面的directBuffer()方法所创建的ByteBuf，其内存并不在堆中，而是占用了操作系统直接管理的空间，Netty对它进行了特殊的处理。其它的方法，创建的ByteBuf都使用“堆”内存。

汇总Unpooled提供的主要方法



方法名称	说明
buffer()	返回 heap ByteBuf
buffer(int initialCapacity)	
buffer(int initialCapacity, int maxCapacity)	
directBuffer()	返回 direct ByteBuf
directBuffer(int initialCapacity)	
directBuffer(int initialCapacity, intmaxCapacity)	
compositeBuffer()	返回 CompositeByteBuf
copiedBuffer()	返回 copied ByteBuf

后面介绍

本质就是进行深复制，得到与原对象一样的副本。

堆缓冲区用法



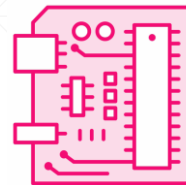
使用`buffer()`方法创建堆缓冲区，从右侧示例可以看到，数据本身放在一个数组中，可以通过获取数组对象的引用，直接修改数组的内容。

`readerIndex`、`writerIndex`等属性，都是直接针对这一内部数组而定义的。

ByteBufCategory.java

```
private static void useHeapByteBuf() {
    ByteBuf heapBuf= Unpooled.buffer(10);
    //写入6个字节
    for (int i = 0; i < 6; i++) {
        heapBuf.writeByte(i);
    }
    //输出: true
    System.out.println(heapBuf.hasArray());
    //获取内部支撑数组的引用
    byte[] array = heapBuf.array();
    //输出支撑数组的内容
    System.out.println(Arrays.toString(array));
    //可以直接修改数组的内容: [0, 1, 2, 3, 4, 5, 0, 0, 0, 0]
    array[1]=100;
    //输出支撑数组的内容: [0, 100, 2, 3, 4, 5, 0, 0, 0, 0]
    System.out.println(Arrays.toString(array));
    //查询ByteBuf的内容
    //      | 0 1 2 3 4 5 6 7 8 9 a b c d e f |
    //+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
    //|00000000| 00 64 02 03 04 05                                     |.d....|
    //+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
    System.out.println(ByteBufUtil.prettyHexDump(heapBuf));
    //查询读写指针位置
    //输出: readerIndex 0
    System.out.println("readerIndex "+heapBuf.readerIndex());
    //输出: writerIndex 6
    System.out.println("writerIndex "+heapBuf.writerIndex());
    //输出: readableBytes 6
    System.out.println("readableBytes "+heapBuf.readableBytes());
}
```

直接缓冲区用法



```
private static void useDirectByteBuf() {  
    ByteBuf directBuf = Unpooled.directBuffer(10);  
    //输出: false  
    System.out.println(directBuf.hasArray());  
    //写入6个字节  
    for (int i = 0; i < 6; i++) {  
        directBuf.writeByte(i);  
    }  
    //输出: readerIndex 0  
    System.out.println("readerIndex " + directBuf.readerIndex());  
    //输出: writerIndex 6  
    System.out.println("writerIndex " + directBuf.writerIndex());  
    //输出: readableBytes 6  
    System.out.println("readableBytes " + directBuf.readableBytes());  
    //从readIndex开始, 读取所有可读内容, 放到字节数组中:  
    byte[] readableBytes = ByteBufUtil.getBytes(directBuf);  
    //输出: [0, 1, 2, 3, 4, 5]  
    System.out.println(Arrays.toString(readableBytes));  
}
```

使用`directBuffer()`方法创建直接缓冲区。

直接缓冲区使用堆外内存, 所以`hasArray()`方法返回`false`。

通常情况下, 会将其数据复制到堆中, 再进行处理。

下一讲



创建好了ByteBuf实例，下一步就是使用它来存取数据了，
下一讲介绍这块内容